

1. Objectif du Sprint & Contexte

Dans le cadre de la refonte responsive de la page d'accueil de notre produit **ClinVCF-OS**, l'objectif de ce sprint est d'intégrer la section de téléchargement située **juste au-dessus du footer**. L'intégration ne doit pas utiliser de valeurs codées en dur (*hardcoded*). Les liens de téléchargement doivent pointer dynamiquement vers les **dernières versions stables (Releases)** hébergées sur le dépôt GitHub public de l'application.

De plus, l'interface doit s'adapter de manière prédictive à l'utilisateur : le système d'exploitation (OS) consultant la page doit être automatiquement détecté depuis le navigateur afin de mettre en surbrillance visuelle le bouton correspondant.

2. Spécifications Fonctionnelles (Cahier des Charges)

- **Ancrage & Navigation** : Un clic sur l'appel à l'action (CTA) principal situé dans le Hero doit déclencher un défilement fluide (*smooth scroll*) positionnant l'écran précisément sur cette zone de téléchargement en bas de page, juste avant le footer.
- **Détection Automatique de l'OS** : Analyse côté client de la plateforme de l'utilisateur afin d'identifier si l'environnement est Windows, macOS ou Linux.
- **Mise en valeur adaptative** : Appliquer une classe de surbrillance CSS spécifique (ex: fond blanc opaque, texte contrasté) sur le bouton de l'OS détecté. Les autres boutons conservent leur état de repos (translucides ou contourés).
- **Récupération Asynchrone** : Interroger l'API REST de GitHub au chargement de la page pour extraire les URLs de téléchargement des fichiers binaires de la toute dernière version disponible.

3. Modélisation Conceptuelle (L'Approche Logique)

AXIOME DE SÉLECTION DE L'ENVIRONNEMENT

Soit Ω l'univers des systèmes d'exploitation grand public définis par l'ensemble :

$$\Omega = \{ \text{Windows, macOS, Linux, Unknown} \}$$

Soit $f: S \rightarrow \Omega$ la fonction de transition qui associe la chaîne brute du User-Agent ou du Navigator Object (S) à un élément unique de Ω . Le comportement visuel attendu est une fonction indicatrice :

$$I(x) = 1 \text{ si } x = f(S), I(x) = 0 \text{ sinon.}$$

Le bouton caractérisé par $I(x) = 1$ recevra la classe CSS active standardisée.

4. Plan d'Action Étape par Étape

Étape 1 : Structuration HTML5 et intégration de l'Ancre

Ajouter un identifiant unique (`id="download-section"`) sur le conteneur principal de la zone de téléchargement placée juste au-dessus du footer. Le bouton d'action du Hero devra pointer vers cet identifiant via son attribut `href`.

Consigne CSS réglementaire : Pour assurer la fluidité du mouvement depuis le Hero jusqu'en bas de page sans surcharger le JavaScript, appliquer la propriété `html { scroll-behavior: smooth; }` dans la feuille de style globale.

Étape 2 : Détection algorithmique du Système d'Exploitation (Vanilla JS)

L'analyse par expressions régulières ou recherche de sous-chaînes dans `navigator.userAgent` permet d'isoler la plateforme de l'utilisateur de manière universelle pour cet entraînement.

```
// Schéma logique de tri
const getOperatingSystem = () => {
  const userAgent = window.navigator.userAgent.toLowerCase();

  if (userAgent.includes('win')) return 'windows';
  if (userAgent.includes('mac')) return 'macos';
  if (userAgent.includes('linux')) return 'linux';

  return 'unknown';
};
```

Étape 3 : Requête de l'API GitHub et Extraction des Données

L'API publique de GitHub expose les données du dernier tag de release via un point de terminaison (endpoint) spécifique accessible en méthode `GET`.

Paramètre	Valeur cible
URL de l'API	<code>https://api.github.com/repos/{owner}/{repo}/releases/latest</code>
Méthode HTTP	<code>GET</code>
Format de Réponse	JSON (JavaScript Object Notation)

Le schéma de réponse renvoyé par GitHub contient un tableau nommé `assets`. Chaque élément de ce tableau représente un fichier compilé disponible au téléchargement. Il convient de filtrer ce tableau pour associer dynamiquement le bon lien de téléchargement au bon bouton.

ÉQUATION DE FILTRAGE DES ASSETS

Pour chaque élément a contenu dans l'ensemble des *Assets*, nous recherchons l'URL de téléchargement U_a telle que l'extension ou le motif du nom de fichier N_a corresponde aux conventions de nommage des binaires cibles :

$$\begin{aligned} \text{Windows} &\rightarrow N_a (.exe \text{ ou } .msi) \\ \text{macOS} &\rightarrow N_a (.dmg \text{ ou } .pkg) \\ \text{Linux} &\rightarrow N_a (.deb, .rpm \text{ ou } .AppImage) \end{aligned}$$

Étape 4 : Manipulation du DOM et Injection des Liens

Une fois les données récupérées par l'intermédiaire d'une promesse (`fetch().then()` ou structure `async/await`), le script doit :

1. Sélectionner les nœuds du DOM correspondants aux boutons de téléchargement à l'aide de `document.querySelector()`.
2. Mettre à jour leur propriété `href` avec l'adresse dynamique récupérée (`asset.browser_download_url`).
3. Ajouter la classe d'activation visuelle (ex: `.btn--highlight`) sur l'élément dont l'identifiant concorde avec la valeur retournée par notre fonction de détection d'OS.

5. Entraînement Pratique (Mise en Situation)

Énoncé de l'entraînement :

Écrire un script JavaScript autonome, à intégrer au bas du fichier HTML ou dans un fichier de script dédié, permettant de réaliser l'arbre décisionnel suivant :

```
SI (Données chargées depuis GitHub avec succès) {
  POUR CHAQUE asset TROUVÉ {
    SI le nom contient ".exe" -> Assigner href au bouton Windows
    SI le nom contient ".dmg" -> Assigner href au bouton macOS
    SI le nom contient ".deb" -> Assigner href au bouton Linux
  }

  CONNAÎTRE l'OS du client
  APPLIQUER le style de surbrillance sur le bouton correspondant à cet OS
} SINON {
  CONSERVER les liens pointant par défaut vers la page d'accueil des releases du projet Git
}
```

Critères de Conformité du Code :

1. Zéro erreur de syntaxe dans la console de développement (F12).
2. Gestion des erreurs réseau incluse (bloc `catch` ou vérification de `response.ok`).
3. Aucune modification des noms de classes CSS déjà définis dans la maquette Figma fournie.